



Arcula: A Secure Hierarchical Deterministic Wallet for Multi-asset Blockchains

Adriano Di Luzio, Danilo Francati^(✉), and Giuseppe Ateniese

Stevens Institute of Technology, Hoboken, NJ, USA
dfrancat@stevens.edu

Abstract. This work presents Arcula, a new design for hierarchical deterministic wallets that brings identity-based public keys to the blockchain. Arcula is built on top of provably secure cryptographic primitives. It generates all its cryptographic secrets from a user-provided seed and enables the derivation of new public keys based on the identities of users, without requiring any secret information. Unlike other wallets, it achieves all these properties while being secure against privilege escalation. We formalize the security model of hierarchical deterministic wallets and prove that an attacker compromising an arbitrary number of users within an Arcula wallet cannot escalate his privileges and compromise users higher in the access hierarchy. Our design works out-of-the-box with any blockchain that enables the verification of signatures on arbitrary messages. We evaluate its usage in a real-world scenario on the Bitcoin Cash network.

Keywords: Hierarchical deterministic wallet · Hierarchical key assignment · Bitcoin · Blockchain

1 Introduction

In recent years, the widespread adoption of distributed financial systems attracted the interest of millions of users, who have exchanged assets and coins in a decentralized and democratic way. Blockchain-based systems like Bitcoin, Ethereum, and Ripple achieve these goals—their users exchange coins by relying on distributed consensus, cryptographic keys, and without depending on any central authority. At the same time, however, the blockchain also attracted the interests of malicious parties that aimed at either compromising the distributed consensus or the cryptographic keys of users. In both cases, their goal is to steal crypto-coins.

This work focuses on this issue and, in particular, on the cryptographic wallets in which users store the cryptographic keys associated with their coins. Similarly to how we store bills and credit cards in our physical wallet, a cryptographic wallet contains the digital keys that allow us to spend crypto-coins on the blockchain (*e.g.*, the Bitcoins that we receive on an address we control).

A good wallet should be secure to external threats – *i.e.*, a physical or remote attacker should not be able to extract keys without the users’ consent – and also resilient to failures and disasters – the users should be able to recover their keys even in case of loss, hardware failure, or natural disaster. We focus, in particular, on a family of cryptographic wallets called hierarchical deterministic wallets (HDWs) that achieve these goals while providing innovative properties. For instance, HDWs allow users to distribute their cryptographic keys according to a hierarchy (reflecting, for example, their different blockchain addresses or the internal organization of a company). In addition, they deterministically generate all their secrets by starting from a user-provided *seed*. As long as users produce the seed, they will be able to recover all their keys. Finally, HDWs allow auditors (external to the wallet and unaware of any of its secrets) to verify the hierarchical relationships between the users and the corresponding keys that they control within the wallet. As we shall see, this last property enables an entirely new set of use cases on the blockchain and in the context of decentralized finance (DeFi).

Currently, HDWs are widely used both by individuals and financial enterprises. Nonetheless, many implementations of HDW are either broken from a security perspective, do not respect the fundamental properties of an HDW, or are exclusively tied to specific blockchains. In this work, we aim at solving this problem. We present Arcula,¹ a new design of HDW that is significantly different from the state of the art. First, Arcula is independent of any particular blockchain: It supports any crypto-system that enables the verification of signatures on arbitrary messages, and we test its implementation in Bitcoin Cash (a fork of the original Bitcoin). Next, we provide a formal proof of its security while respecting all the fundamental properties that an HDW should guarantee. Arcula brings for the first time identity-based addresses to the blockchain: By combining well-tested hierarchical key assignment schemes and certificates based on digital signatures, Arcula enables blockchain users to transmit coins by simply addressing receivers through their usernames or emails. All this while being formally secure against privilege escalation threats and with minimal overhead in the transaction costs. Furthermore, our design of Arcula also enables new capabilities for HDW: For instance, the possibility of dynamically modifying its hierarchy *e.g.*, by adding or removing users, and the possibility of constraining the distribution of the digital keys according to some temporal capabilities (*e.g.*, by distributing keys that will expire at the end of a specified period). Together, Arcula and all these properties unlock new use cases on the blockchain, in decentralized finance, and even for distributed or web-based financial institutions: Making it possible to register distributed loans, distributed promises or futures, and also to handle incoming payments on an entirely untrusted web-server securely.

¹ Arcula is the Latin word to define the small casket where ancient Romans used to store their jewels.

1.1 Our Contributions

We summarize our contributions below.

Syntax and Security model of HDW. We put forward the security definition of HDWs. Intuitively, an HDW should satisfy three properties: First, it deterministically generates its cryptographic keys (associated with users' coins) by starting from an initial seed provided by the user. As long as the user presents the same seed, she will be able to recover her keys, even in case of wallet loss. Second, an HDW also organizes the keys under an access hierarchy that combines groups of users with signing keys. The privileges of users depend on their level in the hierarchy. Users with higher privileges (*i.e.*, higher in the hierarchy), must be able to derive the keys of users on lower levels and, in turn, to sign messages (*i.e.*, transactions) on their behalf. Users on lower levels, however, should not be able to escalate their privileges along the hierarchy, not even when colluding with others. Third, an HDW should permit to generate every public key deterministically (*i.e.*, the address on which coins are transferred) of the wallet by starting from a *master public key*, without requiring any secret information in the process. As we shall see, this requirement enables a set of creative use cases for HDW (*e.g.*, public auditing of blockchain assets, and generation of new keys in untrusted environments). In more detail, let sk_i be the secret signing key of a user v_i of a hierarchy and let pk_i be the corresponding public key. Let sk_j and pk_j be, respectively, the secret and public keys associated with a descendant v_j of v_i (*i.e.*, a user with lower privileges in the hierarchy). We informally summarize the properties as follows:

Property 1 (Deterministic secret derivation). For each descendant v_j of v_i , the secret keys sk_j is deterministically generated by using the secret information of v_i . If v_j has the highest privileges in the hierarchy, then her secret information is generated from a user-provided seed.

Property 2 (Security against privilege escalation). For any set of colluding descendants of v_i , it is computationally infeasible to recover the secret key sk_i of v_i .

Property 3 (Public-key/Address derivation). The public key (address) pk_j of each descendant v_j of v_i is deterministically generated only using public information; the generation process does not require the secret key sk_i or any other secret information.

Arcula from Hierarchical Key Assignment. At its core, Arcula derives the keys of the users by relying on a deterministic version of hierarchical key assignment schemes (HKA): A provably secure process that takes as input a hierarchy of users and assigns a secret key to every user so that users with higher privileges can derive the keys of those below in the hierarchy. To the best of our knowledge, hierarchical key assignment schemes have never been used before to implement an HDW. One of our contributions is to connect these seemingly unrelated fields

of research. Hierarchical key assignment schemes provide several advantages: They are highly efficient and have been extensively studied in the past; they enable Arcula to implement arbitrarily complex hierarchies, integrate temporal capabilities into the wallet, and support the dynamic addition or removal of users to the hierarchy.

Implementation and Evaluation. We implemented Arcula, and we demonstrate that it can be used in the real world by showing how to send and receive funds on the Bitcoin Cash blockchain. Thanks to its design, Arcula is compatible with any blockchains whose scripting language permits the verification of a signature on an arbitrary message (*e.g.*, Ethereum). This is because users must prove ownership of identity-based addresses before spending coins by showing a certificate produced by the wallet’s administrator².

1.2 Applications

HDWs enable different use cases, inspired by both well established and innovative financial applications that specifically tackle the needs of enterprises, governments, and financial institutions.

Enterprises: In enterprises (*e.g.*, financial institutions, or exchanges), the hierarchy of a deterministic wallet might reflect the underlying chain of command or the subdivision in regions, departments, and teams. It allows managers to distribute funds among different branches and ensure fiscal responsibility. In particular, each branch can manage its funds but cannot spend those of other units. The deterministic generation of keys simplifies the management of secrets and guarantees a reliable recovery of the wallet, even in case of catastrophic loss. Cryptocurrency exchanges that manage the keys of hundreds of thousands of users might find this feature particularly useful: Through the HDW, they generate pairs of keys that take into account the hierarchy of users and then rely on the master seed to handle their recovery. Finally, the property of public-key/address derivation enables enterprises to comply with financial laws and regulations without jeopardizing the security of their infrastructure: *E.g.* it allows a (possibly untrusted) auditor to inspect the funds that they hold by deriving all the public keys in the wallet without knowing any secret information.

e-Commerce: An HDW is distinctly beneficial to an e-commerce marketplace. Marketplaces, such as Amazon, typically advertise and sell products to buyers. They also allow third-party vendors to do the same. Cryptocurrencies could help manage the payment flow to these vendors. When selling an item to a buyer, the marketplace generates a fresh payment address for each crypto-coin that it supports. As soon as the buyer transfers the required coins to one of the addresses and the blockchain confirms the transaction, the item gets shipped. The generation of fresh payment addresses leverages the properties of public-key

² In general, the administrator of the wallet is the highest privileged user in the hierarchy.

derivation: Since it does not require any private information, it can take place in an untrusted environment (*e.g.*, a web server exposed to the internet) and allows the e-commerce owner to derive the corresponding secret keys only when spending the funds (*e.g.*, by deriving them offline starting from an intermediate key of the wallet). Even if an attacker compromises the webserver, he will not discover any secret keys, and thus funds received before the attack remain safe and intact. Besides, public-key derivation allows buyers or auditors to check the authenticity of the payment addresses since anyone can generate them from the public key of the marketplace.

Decentralized Finance (DeFi): Decentralized Finance has recently started replacing many of the existing traditional financial tools (*e.g.*, loans, and futures) with open-source alternatives based on the blockchain and its smart contracts. With DeFi, HDW can unleash their full potential: The execution of smart contracts, indeed, cannot rely on any secret information as both the source code and the processing inputs are stored in the clear on the public blockchain. By combining HDW and public derivation, a DeFi smart contract can autonomously derive the public key of the recipient of a transaction (*e.g.*, a user that will receive interests on a loan, or the employee of a company that will receive a percentage of its shares). The process could take place on the blockchain and does not require manual interactions. In turn, HDW and DeFi lay the foundations of a modern, democratic, and decentralized financial world.

1.3 Our Technique

To provide a high-level description of Arcula, we begin by informally describing the security levels in which HDWs operate, ordered by the increasing trust requirements.

Untrusted Environment: This level is entirely untrusted. It refers, for example, to the executing environment of a payee that relies on public-key/address derivation (Property 3) to generate fresh addresses to deliver payments, or to an auditor that, generates the public address of a user (*e.g.*, a department of a company), to inspect the coins held by it.

Hot Environment: This level is semi-trusted. At this stage, the users can access their secret keys and derive those of their descendants. An attacker that compromises a user of the hot environment will compromise, in turn, only her descendants in the hierarchy (Property 2).

Cold Storage: This is the most trusted level of the security model, holding the seed used to generate every key within the wallet deterministically (Property 1). It typically corresponds to an offline location (*e.g.*, a hardware token used to instantiate the wallet) that is physically secured (*e.g.*, in a safe). An attacker that compromises the cold storage has full access to the wallet and every asset it holds.

Figure 1 provides an overview of the design of Arcula, our hierarchical deterministic wallet. Arcula starts from the seed to deterministically generate a master

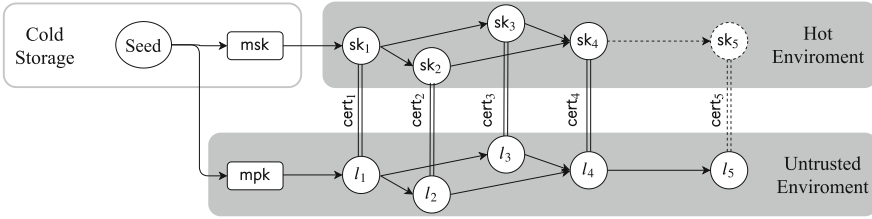


Fig. 1. A glance at the deterministic generation of secrets and identity-based public derivation within Arcula. The users l_1 to l_5 at the bottom of the figure create a hierarchy, encoded as a directed acyclic graph.

pair of secret and public keys ($\mathbf{msk}$, $\mathbf{mpk}$) (Property 1). After the initial instantiation, the seed and the master secret key can be safely moved to cold storage. The master public key uniquely identifies the wallet and will be used in the public key derivation process. In more detail, Arcula generates the public-key/address of i -th user of the hierarchy by concatenating the master public key $\mathbf{mpk}$ and her identity l_i (e.g., a numerical index or a bit string). As a result, Arcula achieves the public-key/address derivation (Property 3), and it can be executed, by design, in an untrusted environment. The master secret key $\mathbf{msk}$ allows, instead, to deterministically generate the secret keys $\mathbf{sk}_i$ corresponding to the users of the hierarchy. As previously mentioned, Arcula (hierarchically) generates the secret keys by leveraging a deterministic hierarchical key assignment. Such primitive assigns a derivation key to every user of the hierarchy, that, in turn, they will use to derive their own secret key $\mathbf{sk}_i$ and those of their descendants (Property 2). Note that in complex hierarchies (e.g., directed acyclic graphs), a user can have two (or multiple) parents (see l_4 and its two parents l_3 and l_2 of Fig. 1). The deterministic hierarchical key assignment allows each parent to compute the secret key of its child without knowing any of the secrets of the other parent. The private key generation should be executed in the context of the hot environment, as compromising a single user will lead to compromising every descendant.

While we have the hierarchical derivation on both the private and public side, the identity-based address and the secret key are completely unrelated. Because of that, Arcula explicitly associates secret keys to their corresponding identity-based addresses through a certificate $\mathbf{cert}_i$ signed by the master secret key $\mathbf{msk}$, that links the identity l_i (top of Fig. 1) to the secret key $\mathbf{sk}_i$ generated by the deterministic key assignment scheme (bottom of Fig. 1). This is done by setting $\mathbf{cert}_i = \text{Sign}_{\mathbf{msk}}(\mathbf{pk}_i || l_i)$ where $\mathbf{pk}_i$ is the real valid public-key of the secret key $\mathbf{sk}_i$. Such a certificate must be revealed whenever a user wants to spend her coins to prove that it is the owner of a particular identity-based address (note that it can be verified just knowing $\mathbf{mpk}$, the master public key of the wallet). Due to these certificates, Arcula can be used only with blockchains whose language permits the verification of signatures (in this case $\mathbf{cert}_i$) on arbitrary messages (the pair $\mathbf{pk}_i || l_i$).

Table 1. Comparison between Arcula and the existing state-of-the-art wallets.

	Security to Privilege Escalation (Property 2)	Public Key Derivation (Property 3)	Deterministic Generation (Property 1)	Hierarchy
BIP32 [17]	No	Yes	Yes	Tree
Hardened BIP32 [17]	Yes	No	Yes	Tree
Gutoski and Stebila [13]	No	Bounded	Yes	Tree
Fan <i>et al.</i> [10]	No	Yes	Yes	Tree
Poulami <i>et al.</i> [6]	—	—	—	No
Goldfeder <i>et al.</i> [12]	—	—	—	No
Gennaro <i>et al.</i> [11]	—	—	—	No
Dikshit and Singh [9]	—	—	—	No
Arcula (Section 5)	Yes	Yes	Bounded	DAG

The identity-based approach that we realized with Arcula provides several advantages: First, it solves the problem of distributing a public key for each user of the hierarchy. Also, it allows for generating an unbounded number of addresses for receiving transactions. On the other hand, Arcula relies on certificates signed by the master secret key to associate the secret key of a user with their identity, to which the transaction was addressed. Users only require these certificates when they sign a transaction for the first time: This means that their creation can be delayed until that moment and that it can occur entirely offline (dashed links sk_5 and $cert_5$ of user 5 in Fig. 1).

2 Related Work

Bitcoin Improvement Proposal 32 (BIP32) defines the state of the art implementation of hierarchical deterministic wallets [17]. In short, let g be the generator point of an Elliptic Curve. A private key sk_i is associated with its public key $pk_i = g^{sk_i}$. Let H be a hash function; the descendants' private keys sk_j and public keys pk_j are defined as:

$$sk_j = H(pk_i || j) + sk_i, \quad pk_j = g^{sk_j} = g^{H(pk_i || j) + sk_i} = g^{H(pk_i || j)} \cdot pk_i \quad (1)$$

Equation (1) satisfies the properties of deterministic generation and public derivation (Properties 1 and 3). However, there is a privilege escalation vulnerability because, by using the descendant private key sk_j and the parent public key pk_i , it is possible to recover the parent private key sk_i by computing $sk_i = sk_j - H(pk_i || j) \bmod q$. In other words, compromising any node leads to compromising the entire wallet. Such attack has been discussed extensively in [4, 5, 13, 17].

BIP32 addresses this issue by designing a *hardened* key derivation method that generates a descendant private key sk_j^h as follows: $\text{sk}_j^h = \text{H}(\text{sk}_i \| i) + \text{sk}_i \bmod q$. The hardened derivation solves the privilege escalation vulnerability but loses the public key derivation (*i.e.*, trades Property 3 for Property 2). Generating a hardened public key pk_j^h now requires the parent secret key sk_i (*i.e.*, $\text{pk}_j^h = g^{\text{sk}_j^h} = g^{\text{H}(\text{sk}_i \| j) + \text{sk}_i} = g^{\text{H}(\text{sk}_i \| j)} \cdot \text{pk}_i$).

In Table 1 we compare Arcula, our hierarchical deterministic wallet, with (hardened) BIP32 and the wallets present in the literature. BIP32 does not satisfy the security to privilege escalation property. The hardened version of BIP32, instead, fails in deriving public keys without requiring additional secrets. Gutoski and Stebila [13] propose an HDW strengthens the security of BIP32. Their design splits each secret key into n shares, distributed to the descendants of the user; reconstructing the secret key requires at least m shares. This solution provides weaker security than Property 2, because m colluding descendants of a user can recover the original secret key (as opposed to preventing any set of colluding descendants from escalating their privileges). In addition, they support Property 3 by publishing the public keys of all the users in the wallet. They do not allow the generation of fresh public keys, and their derivation is bounded to the number of published keys. Fan *et al.* [10] develop an HDW based on Schnorr signatures and trapdoor hash functions that enables the users to sign new transactions without accessing their private keys. A generic user can sign transactions on behalf of its descendants only after authorization by the root of the hierarchy that needs to reveal her the master private trapdoor key. As a result, any authorized user can sign new transactions on behalf of its descendants and all the users of the hierarchy. Compromising a single authorized user leads to revealing every secret stored in the wallet—the cold storage and hot environment of our threat model overlap, and the scheme is not secure against privilege escalation. Poulami *et al.* [6] provide a formal definition of non-hierarchical deterministic wallets and show a set of modifications that make ECDSA-based deterministic wallets provably secure. Goldfeder *et al.* [12] and subsequently Gennaro *et al.* [11] propose a non-hierarchical deterministic wallet where the secret key is shared among n parties, and at least t of them are required to sign a transaction. Dikshit and Singh [9] extend the threshold-based ECDSA signatures to assign different weights to the participants of the protocol. These works deal with non-hierarchical deterministic wallets, and they do not aim at achieving Properties 2 and 3 (depending on the hierarchical structure of the wallet).

Arcula, on the other hand, is the only solution secure against privilege escalation that, at the same time, supports a complex access hierarchy (*e.g.*, a directed acyclic graph (DAG)) and enables unbound public-key/address derivation. However, spending the coins addressed to one of the users requires creating a certificate that associates their identities to their keys. For this reason, Property 1 and, more precisely, spending coins within Arcula, is bound to the generation of a certificate, signed by the master secret key, that authorizes the users.

3 Preliminaries

Notation. Uppercase boldface letters (such as $\mathbf{X}$) are used to denote random variables, lowercase letters (such as x) to denote concrete values, calligraphic letters (such as $\mathcal{X}$) to denote sets, and sans serif letters (such as $\mathbf{A}$) to denote algorithms. Algorithms are modeled as (possibly interactive) Turing machines; if algorithm $\mathbf{A}$ has access to some oracle $\mathbf{O}$, we often write $\mathcal{Q}_{\mathbf{O}}$ for the set of queries asked by $\mathbf{A}$ to $\mathbf{O}$. For a string $x \in \{0, 1\}^*$, we let $|x|$ be its length; $|\mathcal{X}|$ represents the cardinality of the set $\mathcal{X}$. When x is chosen randomly in $\mathcal{X}$, we write $x \leftarrow_{\$} \mathcal{X}$. We write $y = \mathbf{A}(x)$ to denote a run of the algorithm $\mathbf{A}$ on input x and output y ; if $\mathbf{A}$ is randomized, y is a random variable and $\mathbf{A}(x; r)$ denotes a run of $\mathbf{A}$ on input x and (uniform) randomness r . We write $y \leftarrow_{\$} \mathbf{A}(x)$ to denote a run of the randomized algorithm $\mathbf{A}$ over the input x and uniform randomness. An algorithm $\mathbf{A}$ is *probabilistic polynomial-time* (PPT) if $\mathbf{A}$ is randomized and for any input $x, r \in \{0, 1\}^*$ the computation of $\mathbf{A}(x; r)$ terminates in a polynomial number of steps (in the input size). Throughout the paper, we denote by $\lambda \in \mathbb{N}$ the security parameter and we implicitly assume that every algorithm takes as input the security parameter. A function $\nu : \mathbb{N} \rightarrow [0, 1]$ is called *negligible* in the security parameter λ if it vanishes faster than the inverse of any polynomial in λ , i.e., $\nu(\lambda) \in \mathcal{O}(1/p(\lambda))$ for all positive polynomials $p(\lambda)$. We write $\text{negl}(\lambda)$ to denote an unspecified negligible function in the security parameter.

3.1 Signature Scheme

A signature scheme with message space $\mathcal{M}$ is made of the following polynomial-time algorithms.

- $\text{KGen}(1^\lambda)$: The randomized key generation algorithm takes the security parameter and outputs a secret and a public key (sk, pk) .
- $\text{Sign}(\text{sk}, m)$: The randomized signing algorithm takes as input the secret key sk and a message $m \in \mathcal{M}$, and produces a signature σ .
- $\text{Vrfy}(\text{pk}, m, \sigma)$: The deterministic verification algorithm takes as input the public key pk , a message m , and a signature σ , and it returns a decision bit.

A signature scheme is correct if honestly generated signatures always verify correctly.

Definition 1 (Correctness of signatures). A signature scheme $\Pi = (\text{KGen}, \text{Sign}, \text{Vrfy})$ with message space $\mathcal{M}$ is correct if $\forall \lambda \in \mathbb{N}$ and $\forall m \in \mathcal{M}$, we have $\Pr[\text{Vrfy}(\text{pk}, m, \text{Sign}(\text{sk}, m))] = 1$, where $(\text{sk}, \text{pk}) \leftarrow_{\$} \text{KGen}(1^\lambda)$.

For security, we are interested in existential unforgeability, i.e., it must be infeasible to forge a valid signature on a new fresh message.

Definition 2 (Unforgeability of signatures). A signature scheme $\Pi = (\text{KGen}, \text{Sign}, \text{Vrfy})$ is existentially unforgeable under chosen-message attacks if for all PPT adversaries $\mathbf{A}$:

$$\Pr[\mathbf{G}_{\Pi, \mathbf{A}}^{\text{uf}}(\lambda) = 1] \leq \text{negl}(\lambda),$$

where $\mathbf{G}_{\Pi, A}^{\text{euf}}(\lambda)$ is the following experiment:

Setup: The challenger runs $(\text{sk}, \text{pk}) \leftarrow \text{sKGen}(1^\lambda)$ and gives pk to A .

Query: The adversary has access to a signing oracle $\text{O}_{\text{Sign}}(\cdot)$. On input m , the challenger computes and returns $\sigma \leftarrow \text{sSign}(\text{sk}, m)$. Let $\mathcal{Q}_{\text{Sign}}$ denote the messages queried to the signing oracle.

Forgery: The adversary outputs (m, σ) . If $m \notin \mathcal{Q}_{\text{Sign}}$, and $\text{Vrfy}(\text{pk}, m, \sigma) = 1$, output 1, else output 0.

3.2 (Deterministic) Hierarchical Key Assignment Scheme

A hierarchical key assignment scheme [2] assigns a set of cryptographic keys to a set of users in a hierarchy. The hierarchy, encoded as a directed acyclic graph $G = (V, E)$, represents the access rights of users. A path from a node v_i to a node v_j implies that the user v_i has higher privileges than v_j and can assume the same access rights of v_j . We define the set of descendants $\text{Desc}(v_i) = \{v_j \mid v_i \rightsquigarrow_w v_j\}$ of node v_j to be the set of nodes v_j such that there exists a direct path w from v_i to v_j in G . An efficient hierarchical key assignment scheme (HKA) enforces the access hierarchy while minimizing the number of keys distributed to the users.

Typically, HKA schemes assign the cryptographic secrets that they assign at random. Our goal, however, is to leverage an HKA at the core of our wallet, where each secret is deterministically derived from a seed provided by the user. When the hierarchy is static (as in the scope of this paper), fixing the randomness of an HKA algorithm ensures a deterministic generation of keys (the same randomness implies the same key to each node). Within Arcula, we fix the randomness of the HKA developed by Atallah *et al.* [2] to achieve this goal. Besides, in the full version of this work [8], we show how to extend Arcula to handle dynamic hierarchies. In a dynamic context, fixing the randomness does not help since there is no a priori knowledge about the relative position of one node to each other, and, in particular, one cannot make assumptions on the order through which they are added to (or removed from) the hierarchy. For this reason, we propose a modification of the HKA [2] that achieves deterministic generation even with dynamic hierarchies.

A Deterministic Hierarchical Key Assignment (DHKA) scheme with seed space $\mathcal{S}$ is composed of the following polynomial-time algorithms:

Set $(1^\lambda, G, S)$: The deterministic setup algorithm takes as input the security parameter, a DAG $G = (V, E)$, and an initial seed $S \in \mathcal{S}$, and outputs two mappings: 1) a public mapping $\text{Pub} : V \cup E \rightarrow \{0, 1\}^*$, associating a public label l_i to each node v_i in G and a public information y_{ij} to each edge $(v_i, v_j) \in E$; 2) a secret mapping $\text{Sec} : V \rightarrow \{0, 1\}^\lambda \times \{0, 1\}^\lambda$, associating a secret information S_i and a cryptographic key x_i to each node v_i in G . (No secret information is associated to the edges).

Derive $(G, \text{Pub}, v_i, v_j, S_i)$: The deterministic derivation algorithm takes as input the access graph G , the public information Pub , a source node v_i , a target node v_j , and the secret information S_i of node v_i . It outputs the cryptographic key x_j associated to node v_j if $v_j \in \text{Desc}(v_i)$.

The correctness of a DHKA scheme requires that any user v_i should be able to derive, correctly, the secret key x_j of any user $v_j \in \text{Desc}(v_i)$ lower in the hierarchy.

Definition 3 (Correctness of DHKA). A DHKA $\Pi = (\text{Set}, \text{Derive})$ with seed space $\mathcal{S}$ is correct if for every DAG $G = (V, E)$, $\forall \lambda \in \mathbb{N}$, $\forall v_i \in V$, $\forall v_j \in \text{Desc}(v_i)$, $\forall S \in \mathcal{S}$, we have $\Pr[x_j = \text{Derive}(G, \text{Pub}, v_i, v_j, S_i)] = 1$, where $(\text{Pub}, \text{Sec}) = \text{Set}(1^\lambda, G, S)$, $(S_i, x_i) = \text{Sec}(v_i)$, and $(S_j, x_j) = \text{Sec}(v_j)$.

Instead, the security of DHKA requires that even if an attacker corrupts an arbitrary number of descendants of a node, he cannot distinguish its secret key from a uniformly random string. We adapt the definition originally defined by Atallah *et al.* [2] to account for the determinism in our scheme. We define the set of ancestors $\text{Anc}(v_i) = \{v_j \mid v_j \rightsquigarrow_w v_i\}$ of a node v_i to be the set of nodes v_j such that there exists a path w from v_j to v_i in G .

Definition 4 (Key Indistinguishability of DHKA). A DHKA $\Pi = (\text{Set}, \text{Derive})$ with seed space $\mathcal{S}$ is key indistinguishable if for every PPT adversary $\mathcal{A}$ and every DAG $G = (V, E)$:

$$\left| \Pr \left[\mathbf{G}_{\Pi, \mathcal{A}}^{\text{sk-ind}}(\lambda, G) = 1 \right] - \frac{1}{2} \right| \leq \text{negl}(\lambda),$$

where $\mathbf{G}_{\Pi, \mathcal{A}}^{\text{sk-ind}}(\lambda, G)$ is defined in the following way:

- Setup:** The challenger receives a challenge node $v^* \in V$ from the adversary $\mathcal{A}$. The challenger samples $S \leftarrow_{\$} \mathcal{S}$, then runs $\text{Set}(1^\lambda, G, S)$, and gives the resulting public information Pub and G to $\mathcal{A}$. The challenger samples a random bit $b^* \leftarrow_{\$} \{0, 1\}$: If $b^* = 0$, it returns to $\mathcal{A}$ the cryptographic key x_{v^*} associated to node v^* ; otherwise, it returns a random key $\tilde{x}_{v^*}$ of the corresponding length.
- Query:** The adversary has access to a corrupt oracle $\text{O}_{\text{Corr}}(\cdot)$. On input $v_i \notin \text{Anc}(v^*)$, the challenger retrieves $(S_i, x_i) = \text{Sec}(v_i)$ and sends S_i to $\mathcal{A}$.
- Guess:** The adversary outputs a bit $b \in \{0, 1\}$. If $b = b^*$ return 1; otherwise return 0.

4 Hierarchical Deterministic Wallet

A hierarchical deterministic wallet is composed of 5 algorithms ($\text{Set}, \text{DPub}, \text{DPriv}, \text{Sign}, \text{Vrfy}$): 1) Set deterministically instantiates the wallet by generating the public parameters pp and a set of the derivation key $\mathbf{d}_i$, one for user node v_i of the hierarchy. 2) DPriv and DPub are responsible of the derivation of signing and public keys (Properties 1 and 3). DPriv derives the signing key sk_j of a node v_j , descendent of v_i , by using the derivation key $\mathbf{d}_i$ associated to v_i ; DPub derives the corresponding public key pk_j by using the only the public parameters pp . 3) Sign and Vrfy take inspiration from the standard signing and verification algorithms of a digital signature scheme. A hierarchical deterministic wallet $\Pi = (\text{Set}, \text{DPub}, \text{DPriv}, \text{Sign}, \text{Vrfy})$, defined over a seed space $\mathcal{S}$ and message space $\mathcal{M}$ is defined in the following way:

- Set**($1^\lambda, G, S$): The deterministic setup algorithm takes as input a security parameter, an access graph $G = (V, E)$, and an initial seed $S \in \mathcal{S}$, and outputs the public parameters $\mathbf{pp}$ and a set of derivation keys $\{\mathbf{d}_i\}_{v_i \in V}$.
- DPub**($\mathbf{pp}, v_i$): The deterministic public derivation algorithm takes as input the public parameters $\mathbf{pp}$, a target node v_i , and outputs the public key $\mathbf{pk}_i$ associated to node v_i .
- DPriv**($\mathbf{pp}, \mathbf{d}_i, v_i, v_j$): The deterministic private derivation algorithm takes as input the public parameters $\mathbf{pp}$, the derivation key $\mathbf{d}_i$ of node v_i , and a target node $v_j \in \text{Desc}(v_i)$, and outputs the secret key $\mathbf{sk}_j$ associated to node v_j .
- Sign**($\mathbf{sk}_i, m$): The randomized signing algorithm takes as input a message $m \in \mathcal{M}$, and a secret key $\mathbf{sk}_i$, and outputs a signature σ .
- Vrfy**($\mathbf{pk}_i, m, \sigma$): The deterministic verification algorithm takes as input a public key $\mathbf{pk}_i$, a message m , and a signature σ , and outputs a decisional bit b .

A hierarchical deterministic wallet is correct if any user can derive the private and public key of its descendants and create a valid signature on behalf of them. This means that any node v_i can derive the signing key $\mathbf{sk}_j$ of any node $v_j \in \text{Desc}(v_i)$ and produce, in turn, a valid signature σ on behalf of v_j (i.e., that passes the verification process against the public key $\mathbf{pk}_j$ obtained through public key derivation).

Definition 5 (Correctness of HDW). *A hierarchical deterministic wallet $\Pi = (\text{Set}, \text{DPub}, \text{DPriv}, \text{Sign}, \text{Vrfy})$, with seed space $\mathcal{S}$ and message space $\mathcal{M}$, is correct if for every DAG $G = (V, E)$, $\forall v_i, v_j \in V$, $\forall v_j \in \text{Desc}(v_i)$, $\forall S \in \mathcal{S}$, $\forall m \in \mathcal{M}$ the following condition holds:*

$$\Pr [\text{Vrfy}(\mathbf{pk}_j, m, \text{Sign}(\mathbf{sk}_j, m)) = 1] \geq 1 - \text{negl}(\lambda),$$

where $(\mathbf{pp}, \{\mathbf{d}_i\}_{v_i \in V}) = \text{Set}(1^\lambda, G, S)$, $\mathbf{sk}_j = \text{DPriv}(\mathbf{pp}, \mathbf{d}_i, v_i, v_j)$, and $\mathbf{pk}_j = \text{DPub}(\mathbf{pp}, v_j)$.

The security of a hierarchical deterministic wallet draws inspiration from existentially unforgeable signatures. We allow an attacker to corrupt an arbitrary number of users in the hierarchy—by corrupting a user; the attacker implicitly corrupts also all her descendants. In addition, the attacker also has access to a signing oracle that returns signatures on arbitrary messages from any uncorrupted node. We challenge the attacker to forge a signature for a new message on behalf of an uncorrupted node.

Definition 6 (Hierarchical existential unforgeability of HDW). *A hierarchical deterministic wallet is hierarchically existentially unforgeable under chosen-message attacks if for every DAG $G = (V, E)$ and PPT adversary $\mathbf{A}$ the following condition holds:*

$$\Pr [\mathbf{G}_{\Pi, \mathbf{A}}^{\text{heuf}}(\lambda, G) = 1] \leq \text{negl}(\lambda),$$

where experiment $\mathbf{G}_{\Pi, \mathbf{A}}^{\text{heuf}}(\lambda, G)$ is defined in the following way:

Setup: The challenger samples a random $S \leftarrow_{\$} \mathcal{S}$ and executes $(\mathbf{pp}, \{\mathbf{d}_i\}_{v_i \in V}) = \text{Set}(1^\lambda, G, S)$. It gives the public parameters $\mathbf{pp}$ to $\mathbf{A}$.

Query: The adversary A has access to the following oracles:

$O_{\text{Corr}}(\cdot)$: On input $v_i \in V$, the challenger answers by giving $\mathbf{d}_i$ to A . Let $\mathcal{Q}_{\text{Corr}}$ denote the set of nodes v_i that A corrupted, including their descendants $\text{Desc}(v_i)$.

$O_{\text{Sign}}(\cdot, \cdot)$: On input $(m, v_i) \in \mathcal{M} \times V$, the challenger returns $\sigma \leftarrow \text{Sign}_{\text{sk}_i}(m)$ where $\text{sk}_i = \text{DPriv}(\text{pp}, \mathbf{d}_0, v_0, v_i)$. Let $\mathcal{Q}_{\text{Sign}}$ denote the pairs (m, v_i) for which A queried the oracle O_{Sign} .

Forgery: A outputs a forgery (v_i, m, σ) . If $\text{Vrfy}_{\text{pk}_i}(m, \sigma) = 1$ where $\text{pk}_i = \text{DPub}(\text{pp}, v_i)$ and $v_i \notin \mathcal{Q}_{\text{Corr}}$, $(m, v_i) \notin \mathcal{Q}_{\text{Sign}}$, return 1; otherwise return 0.

5 Constructing Arcula from DHKA and Signatures

This section details our construction based on deterministic hierarchical key assignment schemes (DHKA) and digital signatures.

Construction 1 Let $\Gamma = (\text{Set}_\Gamma, \text{Derive}_\Gamma)$ and $\Sigma = (\text{KGen}_\Sigma, \text{Sign}_\Sigma, \text{Vrfy}_\Sigma)$ be respectively a DHKA and a signatures signature scheme. We build Arcula in the following way:

$\text{Set}(1^\lambda, G, S)$: On input the security parameter, a DAG $G = (V, E)$, and a seed $S \in \mathcal{S}$ the algorithm proceeds as follows:

1. Compute $(\text{Pub}, \text{Sec}) = \text{Set}_\Gamma(1^\lambda, G, S)$.
2. For each node $v_i \in V$:
 - (a) Let $(S_i, x_i) = \text{Sec}(v_i)$ and set $\mathbf{d}_i = S_i$.
 - (b) $(\overline{\text{sk}}_i, \overline{\text{pk}}_i) = \text{KGen}_\Sigma(1^\lambda; x_i)$.
3. Output $\text{pp} = (G, \text{Pub}, \{\text{cert}_i\}_{v_i \in V}, \overline{\text{pk}}_0)$ and $\{\mathbf{d}_i\}_{v_i \in V}$ where $\text{cert}_i \leftarrow \text{Sign}_\Sigma(\overline{\text{sk}}_0, (\overline{\text{pk}}_i, l_i))$ for $v_i \in V$, and $l_i = \text{Pub}(v_i)$.³

$\text{DPub}(\text{pp}, v_j)$: On input the public parameters $\text{pp} = (G, \text{Pub}, \{\text{cert}_i\}_{v_i \in V}, \overline{\text{pk}}_0)$ and a node $v_j \in V$, the algorithm returns $\text{pk}_j = (\overline{\text{pk}}_0, l_j)$ where $l_j = \text{Pub}(v_j)$.

$\text{DPriv}(\text{pp}, \mathbf{d}_i, v_i, v_j)$: On input the public parameters $\text{pp} = (G, \text{Pub}, \{\text{cert}_i\}_{v_i \in V}, \overline{\text{pk}}_0)$, the derivation key $\mathbf{d}_i = S_i$, and two nodes $v_i, v_j \in V$ such that $v_j \in \text{Desc}(v_i)$, the algorithm runs $x_j = \text{Derive}_\Gamma(G, \text{Pub}, v_i, v_j, S_i)$ and $(\overline{\text{sk}}_j, \overline{\text{pk}}_j) = \text{KGen}_\Sigma(1^\lambda; x_j)$. Finally, it returns $\text{sk}_j = (\overline{\text{sk}}_j, \overline{\text{pk}}_j, \text{cert}_j)$.

$\text{Sign}(\text{sk}_i, m)$: On input a signing key $\text{sk}_i = (\overline{\text{sk}}_i, \overline{\text{pk}}_i, \text{cert}_i)$ and a message m , the algorithm returns $\sigma = (\overline{\text{pk}}_i, \sigma', \text{cert}_i)$ where $\sigma' \leftarrow \text{Sign}_\Sigma(\overline{\text{sk}}_i, m)$.

$\text{Vrfy}(\text{pk}_i, m, \sigma)$: On input a public key $\text{pk}_i = (\overline{\text{pk}}_0, l_i)$, a message m , and a signature $\sigma = (\overline{\text{pk}}_i, \sigma', \text{cert}_i)$, the algorithm returns 1 if $\text{Vrfy}_\Sigma(\overline{\text{pk}}_0, (\overline{\text{pk}}_i, l_i), \text{cert}_i) = 1$ and $\text{Vrfy}_\Sigma(\text{pk}_i, m, \sigma') = 1$; otherwise it returns 0.

The correctness of the scheme comes directly from the correctness of the underlying primitives. As for security, we establish the following result, whose proof appears in the full version [8]

³ The value l_i is the public label (binary string) associated by the DHKA to the node v_i . Without loss of generality we can assume that $l_i = v_i$ (the public label l_i is just the node number v_i).

Theorem 1. *Let $\Gamma = (\text{Set}_\Gamma, \text{Derive}_\Gamma)$ and $\Sigma = (\text{KGen}_\Sigma, \text{Sign}_\Sigma, \text{Vrfy}_\Sigma)$ be respectively a deterministic hierarchical key assignment and a signature scheme. If Γ is key indistinguishable (Definition 4) and Σ is existentially unforgeable (Definition 2), then the HDW Π from Construction 1 is hierarchically existentially unforgeable (Definition 6).*

Security level of Arcula. Arcula’s public and secrets values must be correctly administered with respect to the HDW architecture model that we defined in Sect. 1.3 (see Fig. 1 for a graphical overview). As mentioned, the *Cold Storage* is where, besides the seed, the master secret key msk must be stored. In Arcula such master secret key correspond to the derivation key d_0 (and the related signing key sk_0) of the root (*i.e.*, $\text{msk} = \text{d}_0$). Indeed, an attacker that compromises d_0 can derive the signing key sk_i of every node, or generates fake certificates that would allow him to spend coins of any identity-based address of the wallet. On the other hand, the *Untrusted Environment* contains any public information that does not put in danger the security of the wallet. Naturally, as depicted in Fig. 1, the master public key mpk lies in this environment since it allows any third party to derive the public-keys/addresses of the hierarchy. Arcula’s master public key is simply the public signing key $\overline{\text{pk}}_0$ of the root (*i.e.*, $\text{mpk} = \overline{\text{pk}}_0$). Indeed, the public derivation requires just the concatenation of $\overline{\text{pk}}_0$ and the identifier l_i of node v_i . All the values $\{(\text{sk}_i, \text{d}_i)\}$ of any node v_i lie in the *Hot Environment*. Compromising the secrets of node v_i leads to compromising all the secrets of its descendants, but none of the other nodes. Lastly, we mention that the certificates $\{\text{cert}_i\}$ they can be published (or even stored on an external server). Even if an adversary knows all the certificates $\{\text{cert}_i\}$ of the wallet, it can not spend the coins of a target user without knowing its secret key $\overline{\text{sk}}_i$.

Extensions. By building Arcula on top of deterministic HKA schemes, our design also inherits some of their more interesting properties. As an example, in the full version of the paper [8], we show: 1) How it is possible to extend Arcula to handle dynamic hierarchies by leveraging the dynamic version of the HKA developed by Atallah *et al.* [2, Section 6] (note that some of the procedures to modify the hierarchy of the HKA proposed in [2, Section 6] are randomized. In order to work with Arcula, we make these procedures deterministic by leveraging a PRF. Intuitively we generate the randomness required by executing the PRF) and, 2) how to incorporate temporal capabilities (*e.g.*, possibility to use cryptographic keys during a given period only) into Arcula by relying on the work of De Santis *et al.* [7].

6 Arcula in the Real World

In this section, we present how Arcula performs in the real world, and we show how to use it to send and receive funds on the Bitcoin Cash blockchain. We stress that Arcula can be used in any blockchain system that enables the verification of signatures of an arbitrary message (*e.g.*, Ethereum) since it requires the verification of a certificate cert_i .

6.1 Technical Implementation

Our open-source implementation of Arcula is available online.⁴ We instantiate the underlying DHKA leveraged by Arcula with the pseudorandom function $F_k(x) = H(k||x)$ (where $H(x)$ is the hash function $\text{SHA3-256}(x)$) and the authenticated AES256 with Galois/Counter Mode (GCM) as the symmetric encryption scheme. We generate a hierarchical deterministic wallet based on the tree defined in BIP43 and BIP44 [14, 15], where the keys to different crypto-coins correspond to different subtrees, and each branch of the subtrees is a chain associated with a single account that contains multiple receiving addresses. We obtain an initial seed S of 512 bits by following the specification of BIP39 [16] that generates a seed from a random mnemonic sequence. We generate the wallet that we use in our tests by fixing the randomness of the mnemonic generation process to the result of the operation `H(correct horse battery staple)`.

6.2 Arcula in Bitcoin Cash

Transactions in Bitcoin Cash. A Bitcoin (Cash) transaction is a cryptographically signed statement that transfers some coins from a sender to a receiver. The sender of the coins signs the transaction through her secret key to spending, in turn, the coins destined to the corresponding public key. Every transaction specifies a locking and an unlocking script. These scripts respectively state the necessary conditions to spend, in a future transaction, the coins being transferred (*i.e.*, their locking condition) and provide the information required to redeem them (*i.e.*, to unlock them as a result of a past transaction). Both scripts are written through a stack-based language that allows simple mathematical operations, stack manipulations and enables simple cryptographic primitives (*i.e.* computing the result of a hash function and verifying a signature).

A typical Bitcoin locking script specifies the address of the receiver (usually through the hash of its public key) and requires him to provide a valid signature to redeem the coins being transferred. More in detail, the locking and unlocking scripts of a standard Bitcoin transaction are defined as follows. Uppercase monospace words indicate operations of the Bitcoin scripting language, while angular brackets enclose variable inputs.

Locking: `OP_DUP OP_HASH160 <H(pk)> OP_EQUALVERIFY OP_CHECKSIG`

Unlocking: `< $\sigma$ > <pk>`

Together, these scripts ensure that the public key pk provided in the unlocking script is the pre-image of the hash $H(\text{pk})$ (the Bitcoin address) contained in the locking script; then, verify the validity of the transaction signature σ under the public key pk .

Every transaction devolves a small amount of fees to the system to incentivize its inclusion in the next block of the chain. Fees are usually measured in coins per byte, and, for this reason, the size of a transaction on the Bitcoin wire protocol

⁴ Available at <https://github.com/aldur/Arcula>.

is directly related to the amount of fees that it should pay to be included in the blockchain. In particular, the length of the locking and unlocking scripts influences directly the final transaction cost. Because of this, BIP16 [1] proposes the pay to script hash mechanism (P2SH), that aims at minimizing the size of the locking script, such that the sender of the transaction will pay its associated fees⁵. The intuition is that instead of specifying the full locking script, the users can constrain the coins of a transaction by locking them to the hash of the original script; then, in the unlocking script, they can provide both the pre-image of the hash, *i.e.*, the full locking script, and its required inputs. Hence, the P2SH locking script is constant (*i.e.*, hash verification) while the unlocking one is the concatenation of the standard locking and unlocking scripts while the locking is constant.

Arcula's Transactions. In Arcula, we identify the nodes of our wallet v_i according to the master public key $\text{mpk} = \overline{\text{pk}}_0$ and to their public label l_i . For this reason, an Arcula address is simply the concatenation of the byte representations of these values that we encode in the locking script. The unlocking script, on the other hand, contains the certificate $\text{cert}_i \leftarrow \text{Sign}_{\Sigma}(\overline{\text{sk}}_0, (\text{pk}_i, l_i))$ and associating the signing public key $\overline{\text{pk}}_i$ to the node v_i with label l_i , and a signature σ of the transaction verifiable through the public signing key $\overline{\text{pk}}_i$. With Arcula, the locking and the unlocking scripts respectively become:

Locking: `OP_DUP OP_TOALTSTACK < l_i > OP_CAT <mpk> OP_CHECKDATASIGVERIFY
OP_FROMALTSTACK OP_CHECKSIG`
Unlocking: `< $\sigma$ > <certi> < $\overline{\text{pk}}_i$ >`

The two scripts: 1) Verify that the certificate cert_i is a valid signature of the message (pk_i, l_i) under the master public key mpk ; 2) verify the validity of the transaction signature σ under the signing public key $\overline{\text{pk}}_i$. In particular, the locking script checks the validity of the certificate cert_i through the operation `OP_CHECKDATASIGVERIFY`, which allows the stack-based scripting language to validate a signature of an arbitrary message (the concatenation of mpk and l_i obtained through the operation `OP_CAT`). To test Arcula, we focus, as an example, on Bitcoin Cash (the original Bitcoin implementation does not support such operation). We first create a transaction that locks 0.5 BCH (the Bitcoin Cash crypto-coin) to a node of our wallet of Sect. 6.1, identified through the master public key mpk (also in the locking script) and the integer label 3. Next, we redeem the coins through a second transaction that provides the transaction signature σ computed using the signing key $\overline{\text{sk}}_i$, an appropriate certificate cert_i signed by the master secret key, and the public signing key $\overline{\text{pk}}_i$ ⁶. We create both the signature and the certificate through the ECDSA signatures scheme on the `secp256k1` elliptic curve used in Bitcoin, and we encode the integer label l_i of the node v_i with 4 bytes.

⁵ Another advantage of P2SH is that it hides the details of the locking script until the users redeem the coins sent by the transaction.

⁶ The transcripts of the transactions are available, respectively, at <https://bit.ly/2UI62tt> and <https://bit.ly/2UoQNGI>.

Table 2. The script bytes sizes of a transaction to a standard Bitcoin address and to an Arcula address. Between brackets the sizes when pay to script hash (P2SH) is in place [1].

Address type	Locking Script	Unlocking Script	Total
Standard (with P2SH)	24 (22)	106 (130)	130 (152)
Arcula (with P2SH)	43 (22)	179 (222)	222 (244)

Transaction Costs. Table 2 reports the sizes, in bytes, of the locking and unlocking scripts of standard Bitcoin transactions and to addresses of our wallet. Every operation of the stack-based scripting language is encoded with a single byte; a standard Bitcoin address is the result of a hash function that outputs 20 bytes; the ECDSA signature and the public key in the unlocking script require, respectively, 73 and 33 bytes. By summing these values up, we find that the locking script of a transaction (without P2SH) to a standard Bitcoin address is 24 bytes long (4 script operations plus the receiver address). In contrast, the unlocking scripts take 106 bytes (the ECDSA signature and its associated public key). In Arcula, on the other hand, the locking script encodes 6 operations, the identifier of a node (that we encode with 4 bytes), and the cold storage public key (33 bytes, as opposed to its 20 bytes hash), for a total of 43 bytes. The unlocking script, instead, contains two ECDSA signatures (one for the transaction and one for the certificate) and the signing public key; as a result, it is 179 bytes long. Overall, the size of the locking and unlocking scripts for a transaction to an Arcula address is 222 bytes, 70% longer than the standard address counterparts. On the other hand, when P2HS is used, the Arcula’s size overhead drops to 60%.

In many cases, the benefits that arise with Arcula justify the increase in the transaction cost. An e-commerce marketplace, as an example, can leverage Arcula’s public key derivation to dynamically derive new addresses (*e.g.*, one for each product of her catalog) in an entirely untrusted environment (*e.g.*, an online web-server) while keeping every signing keys at rest in trusted storage. As a result, the provider obtains the flexibility of handling incoming payments on dynamic addresses and minimizes the risk of losing the coins associated with them. When compared with the financial costs associated with this risk, the additional fees required by the Arcula transactions are negligible. The public key derivation also brings other significant benefits. Many financial regulations require, indeed, companies to be accountable for all the payments that they receive. With Arcula, an auditor can reach this goal by merely inspecting the blockchain while looking for any address that contains the master public key `mpk` that identifies the company. Finally, many enterprises leverage *m-of-n* signatures, where redeeming a transaction requires *m* valid signatures among *n* authorized public keys. Their goal is to enforce the company’s internal structure (*e.g.*, so that either managers or employees can sign transactions) or divide the responsibility of spending coins evenly. The unlocking scripts of *m-of-n* transactions have considerable size: They contain *m* signatures and *n* public keys.

By leveraging Arcula and enforcing an appropriate hierarchy that reflects their internal structure, these companies could reduce the size of the unlocking scripts to only two signatures (the transaction signature and the certificate) and two public keys (the master and signing public keys).

6.3 Optimizations and Compatibility with Bitcoin

The current implementation of Arcula does not require any modification to the underlying protocols and blockchains whose scripting languages allow the verification of signatures of arbitrary messages. Nevertheless, we also propose a set of optimizations that, through minimal modifications to these protocols, reduce both the cost of transactions to Arcula addresses and the amount of storage required on the blockchain. We begin by noting that any authorization certificate cert_i can be used more than once. For this reason, the first optimization that we propose is to *cache* the certificate cert_i as soon as it appears for the first time in an unlocking script. Then, any subsequent transaction signed by $\overline{\text{sk}}_i$ could specify a pointer to the certificate (*e.g.*, with a shorter hash) instead of the certificate itself and, in turn, reduce the size of the unlocking script. As an example, by pointing to the certificate with a 20 bytes hash, we would reduce the size of the Arcula locking and unlocking scripts to be roughly 20 bytes longer than their traditional counterparts. Implementing this optimization requires a new operation in the scripting language to retrieve and verify the certificate.

On the other hand, if we allow for more complex modifications, we can change the signature scheme of the underlying protocols to reduce these space requirements to their optimal value further—a single signature per transaction. Arcula can be implemented with a single signature by leveraging a sanitizable signature scheme [3], *i.e.* a scheme where an authorized party can modify a fraction of the message signed without interacting with the original signer. The intuition is to combine the certificates with the signatures that authorize transactions: Now, the certificate of user v_i that associates her to the signing public key $\overline{\text{pk}}_i$ also includes an additional modifiable portion that will be filled with the transaction details. To spend their coins, the users leverage their sanitizable key to replace the blank transaction with the details they intend to sign.⁷ In their work, Ateniese *et al.* [3] show how to construct a sanitizable signature scheme by combining any signature scheme with a chameleon hash function. This construction would allow Arcula to be used with the traditional Bitcoin blockchain by implementing the sanitizable signatures on top of the ECDSA signature scheme that it already uses. In addition, it would not change the expressiveness of the Bitcoin scripting language: Instead of enabling the verification of signatures on arbitrary messages, it would simply extend the signature verification protocol to account for the certificate embedded in the sanitized signatures.

⁷ The sanitizable keys can be hierarchically deployed by leveraging a second instance of DHKA.

6.4 Unlinkability of Transactions

Individual users of hierarchical deterministic wallets are typically not interested in public key derivation. Unlike enterprises and e-commerce marketplaces, for instance, they simply rely on HDW to recover their keys in case of hardware failure or catastrophic loss. On the other hand, they are often interested in achieving the unlinkability of their transactions, *i.e.* in making sure that multiple transactions sent to their wallet can not be correlated together by an observer that passively monitors the blockchain. However, an HDW that satisfies the public derivation property (Property 3), inevitably reveals the relation between the public keys of the wallet, making it impossible to achieve any privacy notion.

Arcula allows users to trade the public the derivation of public keys in an untrusted setting for the ability to receive payments on uncorrelated pseudonyms. In more detail, these users can ignore the identity-based public derivation that Arcula provides and identify the nodes of the wallet by only their public key $\overline{\text{pk}}_i$. On the blockchain, they can receive standard transactions (costing standard transaction fees) on the public key $\overline{\text{pk}}_i$ and then sign new transactions to redeem the coins through the corresponding private key $\overline{\text{sk}}_i$. Each key pair $(\overline{\text{sk}}_i, \overline{\text{pk}}_i)$ is generated through the underlying DHKA scheme that is secure under key indistinguishability. As a consequence, every two distinct pair $(\overline{\text{sk}}_i, \overline{\text{pk}}_i)$ and $(\overline{\text{sk}}_j, \overline{\text{pk}}_j)$ looks random and independent. As a result, Arcula provides a provably secure alternative to the hardened mode of BIP32: Individual users can generate as many pseudonyms as they need by branching or deepening the DAG that encodes their hierarchy and then leverage the DHKA to generate keys and reliably recover them in case of loss. Note that this modified version of Arcula does not require validation of signatures of arbitrary messages, enabling its usage with any blockchain system, including Bitcoin.

7 Conclusions

In this work, we presented Arcula, a new hierarchical deterministic wallet (HDW) that brings identity-based addresses to the blockchain and is secure against privilege escalation. We leveraged a deterministic hierarchical key assignment (DHKA) scheme to generate the set of cryptographic keys at the core of our wallet. As a result, an attacker that compromises an arbitrary number of users in the hierarchy can not escalate his privileges and compromise other users higher in the hierarchy. Our wallet allows us to dynamically derive new addresses for receiving payments in an entirely untrusted environment, recover every cryptographic key from an initial seed provided by the user, and spend coins on behalf of users lower in the hierarchy. Our design of Arcula considers the legacy and future requirements of modern blockchains. In particular, Arcula is independent of the underlying signature scheme, and it works on top of any protocol that allows the verification of signatures on an arbitrary message (*e.g.*, Bitcoin Cash or Ethereum). For these reasons, we hope that the outcomes of this work will be twofold: To provide the secure and efficient hierarchical deterministic wallet that

we need today and to propose a future-proof design that supports the financial applications and tools of enterprises and companies at scale.

References

1. Andresen, G.: BIP16: pay to script hash (2012). <https://github.com/bitcoin/bips/blob/master/bip-0016.mediawiki>. Accessed 9 Nov 2020
2. Atallah, M.J., Blanton, M., Fazio, N., Frikken, K.B.: Dynamic and efficient key management for access hierarchies. *ACM Trans. Inf. Syst. Secur. (TISSEC)* **12**(3), 1–43 (2009)
3. Ateniese, Giuseppe., Chou, Daniel H., de Medeiros, Breno, Tsudik, Gene: Sanitizable signatures. In: di Vimercati, Sabrina de Capitani, Syverson, Paul, Gollmann, Dieter (eds.) *ESORICS 2005. LNCS*, vol. 3679, pp. 159–177. Springer, Heidelberg (2005). https://doi.org/10.1007/11555827_10
4. Buterin, V.: Deterministic wallets, their advantages and their understated flaws (2013). <https://bitcoinmagazine.com/articles/deterministic-wallets-advantages-flaw-1385450276/>. Accessed 9 Nov 2020
5. Courtois, N.T., Valsorda, F., Emirdag, P.: Private key recovery combination attacks: on extreme fragility of popular bitcoin key management, wallet and cold storage solutions in presence of poor RNG events (2014)
6. Das, P., Faust, S., Loss, J.: A formal treatment of deterministic wallets. In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pp. 651–668 (2019)
7. De Santis, A., Ferrara, A.L., Masucci, B.: New constructions for provably-secure time-bound hierarchical key assignment schemes. *Theor. Comput. Sci.* **407**(1–3), 213–230 (2008)
8. Di Luzio, A., Francati, D., Ateniese, G.: Arcula: A secure hierarchical deterministic wallet for multi-asset blockchains. *arXiv preprint arXiv:1906.05919* (2019)
9. Dikshit, P., Singh, K.: Efficient weighted threshold ECDSA for securing bitcoin wallet. In: *2017 ISEA Asia Security and Privacy (ISEASP)*, pp. 1–9. IEEE (2017)
10. Fan, Chun-I., Tseng, Yi-Fan., Su, Hui-Po., Hsu, Ruei-Hau, Kikuchi, Hiroaki: Secure hierarchical bitcoin wallet scheme against privilege escalation attacks. *Int. J. Inf. Secur.* **19**(3), 1–11 (2019). <https://doi.org/10.1007/s10207-019-00476-5>
11. Gennaro, Rosario., Goldfeder, Steven, Narayanan, Arvind: Threshold-optimal DSA/ECDSA signatures and an application to bitcoin wallet security. In: Manulis, Mark, Sadeghi, Ahmad-Reza, Schneider, Steve (eds.) *ACNS 2016. LNCS*, vol. 9696, pp. 156–174. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-39555-5_9
12. Goldfeder, S., et al.: Securing bitcoin wallets via a new DSA/ECDSA threshold signature scheme (2015)
13. Gutoski, Gus, Stebila, Douglas: Hierarchical deterministic bitcoin wallets that tolerate key leakage. In: Böhme, Rainer, Okamoto, Tatsuaki (eds.) *FC 2015. LNCS*, vol. 8975, pp. 497–504. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-47854-7_31
14. Palatinus, M., Rusnak, P.: BIP43: purpose field for deterministic wallets (2014). <https://github.com/bitcoin/bips/blob/master/bip-0043.mediawiki>. Accessed 9 Nov 2020
15. Palatinus, M., Rusnak, P.: BIP44: multi-account hierarchy for deterministic wallets (2014). <https://github.com/bitcoin/bips/blob/master/bip-0044.mediawiki>. Accessed 9 Nov 2020

16. Palatinus, M., Rusnak, P., Voisine, A., Bowe, S.: BIP39: Mnemonic code for generating deterministic keys (2013). <https://github.com/bitcoin/bips/blob/master/bip-0039.mediawiki>. Accessed 9 Nov 2020
17. Wuille, P.: BIP32: Hierarchical deterministic wallets (2012). <https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki>. Accessed 9 Nov 2020